



ASP.NET CORE ESSENTIALS

Module 0-5

Table of Contents

Module 0: History and Evolution of .NET to .NET Core	4
0.1 The Birth of .NET Framework.....	4
0.2 Introduction of ASP.NET.....	4
0.3 Limitations of .NET Framework.....	5
0.4 Introduction of .NET Core.....	5
0.5 Evolution to .NET 5, 6, and 7	5
0.6 Key Differences Between .NET Framework and .NET Core	5
0.7 Why ASP.NET Core Matters Today.....	6
Module 1: Introduction to ASP.NET Core MVC.....	7
1.1 What is ASP.NET Core?	7
1.2 Understanding the MVC Architecture.....	7
1.2.1 Model.....	7
1.2.2 View	8
1.2.3 Controller	8
1.3 The Request-Response Lifecycle	8
Step-by-step flow.....	9
1.4 ASP.NET Core Project Structure.....	10
Key folders explained	10
1.5 Understanding Program.cs.....	11
1.6 Creating Your First MVC Controller	11
1.7 Creating Your First View	12
1.8 Clean Separation of Concerns	12
1.9 Notes and Best Practices.....	12
1.10 Summary	13
Module 2: Routing in ASP.NET Core MVC.....	14
2.1 What Is Routing?	14
2.2 Conventional Routing.....	14
Explanation of the pattern	15
How it behaves.....	15
Example Controller	15
2.3 Optional Parameters in Routes.....	15
2.4 Default Values in Routes.....	16
2.5 Route Constraints	16

2.6 Attribute Routing	17
2.7 Attribute Routing Variants	17
2.7.1 Simple Route.....	17
2.7.2 Using Tokens in Routing.....	18
2.7.3 Multiple Routes for One Action.....	18
2.7.4 Attribute Routing with Constraints.....	18
2.8 Combining Conventional + Attribute Routing	19
2.9 Route Priority and Ambiguity	19
2.10 Understanding route precedence	19
2.11 Summary of Module 2	20
Module 3: Views and Razor Syntax	21
3.1 What Is a View in MVC?	21
3.2 How Views Are Returned from Controllers	21
3.3 Razor Syntax Basics	22
3.3.1 Output C# variables.....	22
3.3.2 C# code blocks.....	22
3.3.3 If conditions.....	22
3.3.4 Loops.....	22
3.3.5 HTML inside C# block.....	22
3.4 Strongly-Typed Views	23
3.5 Passing Data Using ViewBag and ViewData	23
ViewBag (Dynamic object).....	23
ViewData (Dictionary).....	24
Best Practice.....	24
3.6 Layouts in MVC	24
3.7 Using a Layout in a View	25
3.8 Sections in Views	25
3.9 Partial Views	25
3.10 View Components	26
3.11 Tag Helpers	26
3.11.1 Anchor Tag Helper.....	27
3.11.2 Form Tag Helper.....	27
3.11.3 Input Tag Helper.....	27
3.12 Rendering Raw HTML	27

3.13 Error Handling in Views.....	27
3.14 Best Practices for Views	27
3.15 Summary.....	28
Module 4: Model Binding & Validation.....	29
4.1 What Is Model Binding?.....	29
Example: Basic Model Binding.....	29
4.2 Binding to Complex Objects	29
Example: Student Model.....	30
4.3 Sources of Model Data	30
Attributes for Explicit Binding.....	30
4.4 Introduction to Validation	31
4.4.1 Common Data Annotations.....	31
4.4.2 Example: Validating a Model	31
4.5 Displaying Validation Errors in Views	32
4.6 Client-Side Validation.....	32
4.7 ModelState and Validation Workflow	33
4.8 Custom Validation	33
4.9 Summary of Model Binding & Validation	34
Module 5: Custom Validation in ASP.NET Core MVC	35
5.1 Why Use Custom Validation?.....	35
5.2 Creating a Custom Validation Attribute	35
Example 1: Future Date Validation	35
5.3 Conditional Validation	36
Example 2: Conditional Required	36
5.4 Validation Against Database or External Sources	37
Example 3: Unique Email Validation.....	37
5.5 Integrating Custom Validation in MVC.....	38
5.6 Client-Side Support for Custom Validation.....	39
5.7 Best Practices for Custom Validation.....	39
5.8 Summary of Custom Validation.....	39

Module 0: History and Evolution of .NET to .NET Core

Before diving into ASP.NET Core development, it is important to understand the **history of .NET** and why **.NET Core** was created. Knowing this provides perspective on **platform choices, architecture, and future-proof development**.

0.1 The Birth of .NET Framework

- **Year:** 2002
- **Creator:** Microsoft
- **Purpose:** Simplify Windows application development
- **Key Features:**
 - **CLR (Common Language Runtime):** Executes code and provides memory management
 - **BCL (Base Class Library):** Prebuilt classes for file I/O, collections, networking, etc.
 - **Windows-centric:** Initially only for Windows OS
- **Languages Supported:** C#, VB.NET, F#

.NET Framework allowed developers to build **desktop applications, web applications (ASP.NET Web Forms), and server apps** efficiently on Windows.

0.2 Introduction of ASP.NET

- **Year:** 2002
- **Part of .NET Framework**
- **Purpose:** Build web applications using a Windows server
- **Key Points:**
 - Initially **Web Forms** – event-driven, drag-and-drop UI
 - Later, **ASP.NET MVC** introduced (2009) – Model-View-Controller pattern
 - Allowed separation of **business logic, UI, and data**

While powerful, ASP.NET was **tied to Windows**, making cross-platform deployment impossible.

0.3 Limitations of .NET Framework

1. **Windows-only:** Could not run on Linux or macOS
2. **Monolithic:** Large runtime, difficult to scale microservices
3. **Slow adoption for cloud-native apps**
4. **Limited modularity:** Hard to include only required components

These limitations prompted Microsoft to **rethink the platform** for modern web and cloud apps.

0.4 Introduction of .NET Core

- **Year:** 2016
- **Purpose:** Modern, modular, cross-platform framework
- **Key Features:**
 - **Cross-platform:** Windows, Linux, macOS
 - **High performance:** Lightweight and fast
 - **Open-source:** Hosted on GitHub, community-driven
 - **Modular design:** NuGet packages for specific features
 - **Side-by-side versions:** Multiple apps can run with different .NET Core versions

.NET Core was designed for **cloud-native, microservices, and cross-platform web development**, addressing the limitations of .NET Framework.

0.5 Evolution to .NET 5, 6, and 7

- **.NET 5 (2020):** Unified .NET Framework and .NET Core into a single platform
- **.NET 6 (2021) & .NET 7 (2022):** Long-term support (LTS), improved performance, minimal APIs, and simplified development
- **ASP.NET Core:** Modern web framework built on top of .NET Core / .NET 5+
 - Supports **MVC, Web API, Razor Pages, SignalR**
 - Optimized for **cross-platform, cloud, and containerized applications**

Modern .NET is fully **open-source, high-performance, and flexible**, enabling developers to build applications for **desktop, web, mobile, cloud, and IoT**.

0.6 Key Differences Between .NET Framework and .NET Core

Feature	.NET Framework	.NET Core / .NET 5+
Platform	Windows only	Cross-platform
Open-source	No	Yes
Performance	Moderate	High-performance
Deployment	System-wide	Side-by-side per app
Cloud-ready	Limited	Optimized for cloud
API/Web Framework	ASP.NET, Web Forms	ASP.NET Core (MVC, API)

Summary:

- **.NET Framework:** Legacy, Windows-bound, monolithic
- **.NET Core / .NET 5+:** Modern, cross-platform, cloud-native, modular

0.7 Why ASP.NET Core Matters Today

1. **Cross-platform** – build once, run anywhere
2. **Cloud-native** – designed for microservices, containers, Kubernetes
3. **High performance** – faster than classic ASP.NET
4. **Unified programming model** – MVC, Web API, and Razor Pages in one framework
5. **Open-source and community-driven** – constant improvements

Understanding this history helps learners appreciate **why ASP.NET Core was built, its design decisions, and how it differs from legacy ASP.NET.**

Module 1: Introduction to ASP.NET Core MVC

1.1 What is ASP.NET Core?

ASP.NET Core is a modern, open-source, cross-platform framework developed by Microsoft for building web applications and APIs. Unlike the older ASP.NET Framework—which ran only on Windows—ASP.NET Core can run on **Windows, Linux, and macOS**, making it truly flexible and cloud-ready.

The framework is:

- **High-performance** (consistently among the fastest web frameworks globally)
- **Modular** (you add only the packages you need)
- **Lightweight**
- **Built with dependency injection support**
- **Designed for modern web development**, including REST APIs, microservices, and server-side rendering

ASP.NET Core is the foundation for multiple application types, including:

- MVC Web Apps
- Razor Pages
- REST APIs
- Minimal APIs
- Real-time apps via SignalR
- gRPC services

This book focuses on **ASP.NET Core MVC** and **ASP.NET Core Web API**, both essential for real-world applications.

1.2 Understanding the MVC Architecture

MVC stands for **Model–View–Controller**, a design pattern used to separate responsibilities inside a web application. This separation makes the application easier to maintain, test, and extend.

1.2.1 Model

The Model represents **data and business rules**.

Examples:

- A `Student` class that holds student information
- ViewModels or DTOs used to transfer data
- Database entities

Models do **not** contain UI logic.

1.2.2 View

The View represents the **user interface**. In ASP.NET Core MVC, Views are written using **Razor**, a combination of HTML and C#.

Views display:

- Form input fields
- Tables
- Lists
- Alerts
- Layouts and partial views

Views do **not** perform data processing.

1.2.3 Controller

A Controller handles **incoming requests**, processes data (possibly with help from Models), and returns a **response**.

A response can be:

- A View
- JSON data
- File download
- Redirect
- Status code

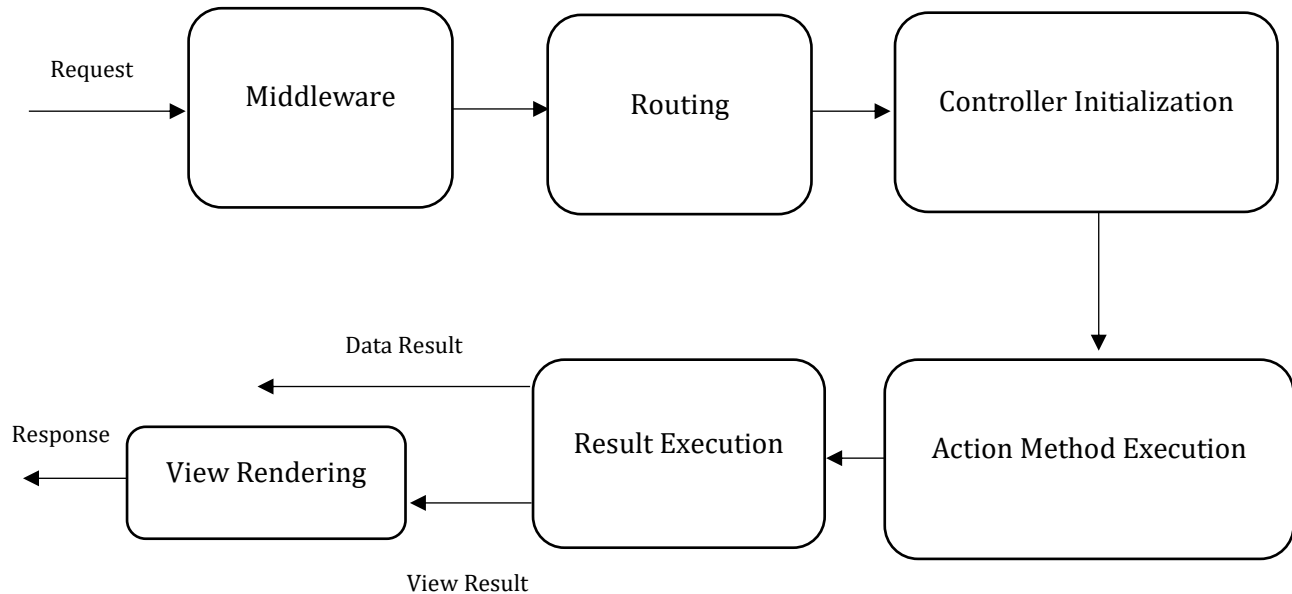
Example:

```
public IActionResult Index()
{
    return View();
}
```

The controller decides **what** to send; the view decides **how** to present it.

1.3 The Request–Response Lifecycle

To understand how MVC works, it is important to understand what happens when the user types a URL into the browser.



Step-by-step flow

1. The browser sends a request to the server
2. ASP.NET Core receives the request through the **Kestrel web server**
3. The request passes through the **middleware pipeline** (authentication, routing, etc.)
4. Routing identifies the appropriate Controller and Action
5. The Controller executes the Action
6. The Action may interact with:
 - o Models
 - o Database
 - o Services
7. The Action returns a response (View or JSON)
8. The browser displays the final output

This lifecycle helps you understand where things go wrong when debugging.

1.4 ASP.NET Core Project Structure

When you create a new MVC project, the default folder structure includes:

```
Controllers/  
Views/  
Models/  
wwwroot/  
appsettings.json  
Program.cs
```

Key folders explained

Controllers

Contains all controllers.

Example: HomeController.cs [Controllers should have suffix Controller]

Views

Contains View files organized by controller name.

Example:

```
Views/Home/Index.cshtml  
Views/Shared/_Layout.cshtml
```

Models

Holds Model classes—entities, DTOs, ViewModels, etc.

wwwroot

This is the public static files directory.

Contains:

- CSS
- JavaScript
- Images
- Fonts

Anything inside `wwwroot` is directly accessible by the browser.

1.5 Understanding Program.cs

Earlier ASP.NET versions used `Startup.cs`.
Now everything is configured inside **Program.cs**.

This file:

- Configures services (Dependency Injection)
- Configures MVC
- Configures routing
- Builds and runs the application

Example snippet:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}"
);

app.Run();
```

This “registration before usage” style defines the application behavior.

1.6 Creating Your First MVC Controller

A basic controller looks like this:

```
public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

ASP.NET Core automatically looks for:

Views/Home/Index.cshtml

This is called **convention over configuration**—the framework handles standard patterns automatically.

1.7 Creating Your First View

Inside Views/Home/Index.cshtml:

```
<h1>Welcome to ASP.NET Core MVC</h1>
<p>This is your first MVC view.</p>
```

When you run the project, you will see this page rendered.

1.8 Clean Separation of Concerns

One of the biggest strengths of MVC is separation:

- **Controllers** → Handle requests
- **Models** → Store and validate data
- **Views** → Render UI

This makes your project:

- Easier to maintain
 - Easier to test
 - Easier to scale
 - More structured for large teams
-

1.9 Notes and Best Practices

- Keep Controllers thin; move logic to services or BLL
 - Keep Models clean and simple
 - Use ViewModels instead of sending database entities to Views
 - Use appsettings.json for configuration
 - Follow naming conventions (very important for MVC routing)
-

1.10 Summary

In this introductory module, you learned:

- What ASP.NET Core is and why it's powerful
- The MVC pattern and how it structures an application
- How the request-response lifecycle works
- The purpose of each project folder
- How to create your first Controller and View
- The importance of separation of concerns

You now have the foundation needed to explore routing, razor views, model binding, and more—all covered in the upcoming modules.

Module 2: Routing in ASP.NET Core MVC

Routing is one of the most fundamental components of an MVC application. Without routing, the framework would have no idea which controller or action should handle a particular URL request. In ASP.NET Core MVC, routing is powerful, flexible, and highly extensible.

This chapter will help you understand how routing works, how to configure it, and how to apply it in real projects.

2.1 What Is Routing?

Routing is the mechanism that **maps a URL** coming from the browser to a **specific controller and action**.

Example URL:

`https://example.com/products/details/5`

Routing determines:

- **Controller:** ProductsController
- **Action:** Details
- **Parameter:** id = 5

Without proper routing, the above request would simply fail.

ASP.NET Core provides two routing styles:

1. **Conventional Routing** (pattern-based)
2. **Attribute Routing** (controller/action-level)

Both have their uses, and you will learn how to use them effectively.

2.2 Conventional Routing

This is the default routing style used in MVC applications. It follows a predefined URL pattern.

The default routing pattern looks like this:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Explanation of the pattern

- {controller=Home} → default controller is **Home**
- {action=Index} → default action is **Index**
- {id?} → optional parameter id

How it behaves

URL → Controller/Action mapping:

URL	Controller	Action	id
/	Home	Index	-
/home/about	Home	About	-
/products/list	Products	List	-
/products/details/10	Products	Details	10

This is called **convention over configuration**.

Example Controller

```
public class ProductsController : Controller
{
    public IActionResult Details(int id)
    {
        return Content($"Product ID = {id}");
    }
}
```

2.3 Optional Parameters in Routes

Optional parameters use the ? symbol.

pattern: "{controller=Home}/{action=Index}/{id?}"

Example:

- /products/details → id not required
- /products/details/20 → id = 20

You can also add *multiple* optional parameters:

```
pattern: "{controller}/{action}/{category?}/{page?}"
```

2.4 Default Values in Routes

You can define default values inside the route pattern.

Example:

```
pattern: "shop/{action=List}/{id?}";
```

This means:

- Controller is always **ShopController**
- If no action → use **List**
- If no id → null

URL Behaviors:

URL	Action	id
/shop	List	-
/shop/details/3	Details	3

2.5 Route Constraints

Route constraints restrict which URLs are valid.

Example using **integer constraint**:

```
pattern: "{controller=Home}/{action=Index}/{id:int?}";
```

This ensures:

- /products/details/5 → valid
- /products/details/abc → 404 Not Found

Other common constraints:

Constraint	Meaning
int	Only integers
bool	true/false
datetime	Valid date
min(5)	Minimum value
max(10)	Maximum
alpha	Letters only

Example:

```
pattern: "blog/{year:int}/{month:int:min(1):max(12)}";
```

2.6 Attribute Routing

Instead of configuring routing globally, you can configure route patterns **inside controllers** using attributes.

Example:

```
[Route("products")]
public class ProductsController : Controller
{
    [Route("details/{id}")]
    public IActionResult Details(int id)
    {
        return View();
    }
}
```

Now the URL becomes:

```
/products/details/5
```

2.7 Attribute Routing Variants

2.7.1 Simple Route

```
[Route("about")]
public IActionResult About() => View();
```

→ URL: /about

2.7.2 Using Tokens in Routing

Tokens let you reuse controller/action names.

```
[Route("[controller]/[action]")]
public class StudentsController : Controller
{
    public IActionResult List() => View();
}
```

→ URL becomes:

/students/list

2.7.3 Multiple Routes for One Action

```
[Route("products/details/{id}")]
[Route("item/{id}")]
public IActionResult Details(int id)
{
    return View();
}
```

→ URLs:

- /products/details/5
 - /item/5
-

2.7.4 Attribute Routing with Constraints

```
[Route("products/details/{id:int:min(1)}")]
public IActionResult Details(int id)
{
    return View();
}
```

Only allows positive integers.

2.8 Combining Conventional + Attribute Routing

You can use both simultaneously:

- **Use conventional routing** for general MVC pages
- **Use attribute routing** for API-like URLs or SEO-friendly URLs

Example:

- `/products/details/10` → convention
- `/products/special/10` → attribute

This combination is common in real-world applications.

2.9 Route Priority and Ambiguity

When using both routing styles:

1. **Attribute routing always wins**
2. MVC tries to pick the most specific match
3. Ambiguities will throw runtime errors

Example of conflicting patterns:

```
[Route("products/{id}")]
[Route("products/details/{id}")]
```

If not handled carefully, one will shadow the other.

2.10 Understanding route precedence

Most specific route wins:

1. `/products/details/5` (attribute)
2. `/products/{id}` (attribute)
3. `{controller}/{action}/{id?}` (convention)

ASP.NET Core evaluates them in that order.

2.11 Summary of Module 2

In this chapter, you learned:

- What routing is and how it maps URLs to controllers
- How conventional routing works using patterns
- How to define optional parameters and defaults
- How route constraints validate URLs
- How attribute routing can provide cleaner, SEO-friendly URLs
- How to combine routing strategies
- How route precedence works and how to avoid ambiguity

Routing is a core skill for every MVC developer. With this knowledge, you can now build clean, predictable URL structures for your application.

Module 3: Views and Razor Syntax

Views are the presentation layer of your MVC application. If the Controller is the brain and the Model is the structure of data, then the View is the face of the application—the part that users directly interact with.

In ASP.NET Core MVC, views are created using **Razor**, a powerful templating engine that combines HTML with C# in a clean and readable way.

This chapter teaches you everything you need to know about Views, Razor syntax, layouts, partials, sections, and best practices.

3.1 What Is a View in MVC?

A View is a file that contains:

- **HTML** → For structure
- **Razor syntax** → For dynamic data
- **Tag Helpers** → For generating URLs and forms
- **C# expressions** → For rendering logic

Views do not contain business logic or database calls.

A typical View file:

Views/Home/Index.cshtml

The extension **.cshtml** stands for *C# + HTML*, which is the heart of the Razor engine.

3.2 How Views Are Returned from Controllers

Controller Action:

```
public IActionResult Index()
{
    return View();
}
```

ASP.NET Core will automatically look for:

Views/Home/Index.cshtml

You can explicitly specify a different view:

```
return View("MyCustomView");
```

3.3 Razor Syntax Basics

Razor syntax starts with @.

3.3.1 Output C# variables

```
@DateTime.Now  
@Model.Name  
@ViewBag.Message
```

3.3.2 C# code blocks

```
@{  
    var message = "Welcome to Razor!";  
}  
<p>@message</p>
```

3.3.3 If conditions

```
@if (Model.IsActive)  
{  
    <p>The account is active.</p>  
}  
else  
{  
    <p>Inactive account.</p>  
}
```

3.3.4 Loops

```
@foreach (var item in Model.Students)  
{  
    <li>@item.Name</li>  
}
```

3.3.5 HTML inside C# block

```
@{  
    <p>This is rendered inside a block.</p>
```

```
}
```

Razor intelligently switches between HTML and C#.

3.4 Strongly-Typed Views

Views can directly receive a **Model**.

Example model:

```
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Controller:

```
public IActionResult Details()
{
    var student = new Student { Name = "John", Age = 22 };
    return View(student);
}
```

View:

```
@model Student

<h2>@Model.Name</h2>
<p>Age: @Model.Age</p>
```

This is the preferred way to pass data to views—clean, strongly typed, and compiler-verified.

3.5 Passing Data Using ViewBag and ViewData

ViewBag (Dynamic object)

```
ViewBag.Message = "Hello!";
```

View:


```
<p>@ViewBag.Message</p>
```

ViewData (Dictionary)

```
ViewData["Title"] = "Dashboard";
```

View:

```
<h1>@ViewData["Title"]</h1>
```

Best Practice

Use **strongly-typed models** for real data.

Use ViewBag/ViewData for small UI messages.

3.6 Layouts in MVC

A layout is like a master page that defines your shared UI structure (header, footer, menu).

Default layout:

```
Views/Shared/_Layout.cshtml
```

A typical layout contains:

```
<!DOCTYPE html>
<html>
<head>
    <title>@ViewData["Title"] - MyApp</title>
</head>
<body>
    <header>My App Header</header>

    <div class="container">
        @RenderBody()
    </div>

    <footer>My Footer</footer>
</body>
</html>
```

@RenderBody() tells MVC where to place the content of each page.

3.7 Using a Layout in a View

At the top of your View:

```
@{
    Layout = "_Layout";
}
```

or to disable layout:

```
@{
    Layout = null;
}
```

3.8 Sections in Views

Sections allow a page to send additional content to the layout.

Layout:

```
@RenderSection("Scripts", required: false)
```

View:

```
@section Scripts {
    <script>alert("Loaded!");</script>
}
```

3.9 Partial Views

A Partial View is a reusable UI component.

Examples:

- Navbar
- Sidebar
- Footer
- Table row template

Create a partial (underscore prefix is recommended):

Views/Shared/_StudentCard.cshtml

Usage:

```
@model Student
```

```
<div class="card">
    <h4>@Model.Name</h4>
    <p>Age: @Model.Age</p>
</div>
```

Render Partial in a View:

```
@await Html.PartialAsync("_StudentCard", student)
```

or

```
<partial name="_StudentCard" model="student" />
```

3.10 View Components

View Components are like mini-controllers for reusable UI + logic.

Example use case:

- Sidebar showing latest posts
- Shopping cart summary
- Notifications box

Basic structure:

- A class in ViewComponents folder
- A view in Views/Shared/Components/<Name>/Default.cshtml

They are advanced; you can use partials until you need them.

3.11 Tag Helpers

Tag Helpers are one of the biggest advantages of ASP.NET Core compared to old MVC.

They look like HTML but generate server-side logic.

Examples:

3.11.1 Anchor Tag Helper

```
<a asp-controller="Home" asp-action="Index">Home</a>
```

Generates correct URL automatically.

3.11.2 Form Tag Helper

```
<form asp-controller="Account" asp-action="Login" method="post">
```

3.11.3 Input Tag Helper

```
<input asp-for="Name" class="form-control" />
```

3.12 Rendering Raw HTML

If the model contains HTML content:

```
@Html.Raw(Model.Description)
```

Be careful with user-generated content to avoid XSS attacks.

3.13 Error Handling in Views

If a model property is missing and you try to print it:

```
@Model.SomeMissingProperty
```

The view renders a descriptive error.
That's why compiler-checked models are recommended.

3.14 Best Practices for Views

- ✓ Use **strongly-typed models** instead of ViewBag/ViewData
- ✓ Keep the View clean—no business logic
- ✓ Use partials for repeated UI pieces
- ✓ Use tag helpers instead of hard-coded links

- ✓ Keep views focused on presentation only
 - ✓ Avoid heavy loops or calculations in views
 - ✓ Store static files in `wwwroot`
-

3.15 Summary

In this chapter, you learned:

- How views work in MVC
- How Razor syntax mixes C# and HTML
- How to use strongly-typed models
- Differences between ViewBag, ViewData, and Model
- Layout pages, partial views, and sections
- Tag Helpers and how they simplify HTML
- Best practices for writing clean, maintainable views

With this foundation, you are ready for the next modules—**Model Binding, Validation**, and **Database Integration**—which introduce dynamic pages, forms, and input processing.

Module 4: Model Binding & Validation

Model binding and validation are core concepts in ASP.NET Core MVC. They make web applications powerful, secure, and easy to maintain. By understanding these concepts, you can efficiently process user input, validate it, and protect your application from invalid or malicious data.

4.1 What Is Model Binding?

Model binding is the process by which **ASP.NET Core automatically maps HTTP request data to action method parameters** or model properties.

When a user submits a form or sends a query string, ASP.NET Core can:

- Map form inputs (POST)
- Map query string values (GET)
- Map route values (URL)
- Bind JSON payloads for APIs

Example: Basic Model Binding

Controller Action:

```
public IActionResult Greet(string name)
{
    ...
    return View();
}
```

Request:

/Home/Greet?name=John

Output:

John will be mapped with name variable

ASP.NET Core automatically binds the name query string to the name parameter.

4.2 Binding to Complex Objects

You can bind entire models instead of individual parameters.

Example: Student Model

```
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Controller:

```
[HttpPost]
public IActionResult Create(Student student)
{
    ...
    return View();
}
```

Form (HTML):

```
<form method="post" action="/Home/Create">
    <input type="text" name="Name" />
    <input type="number" name="Age" />
    <button type="submit">Submit</button>
</form>
```

ASP.NET Core automatically maps Name and Age to the student object.

4.3 Sources of Model Data

Model binding can pull data from multiple sources:

Source	Example
Form Data	<input name="Name" />
Query String	/Home/Greet?name=John
Route Values	/Products/Details/5 → id=5
JSON Body	API POST request
Headers	Custom header values
Cookies	Request.Cookies["Token"]
Services	Using [FromServices] for DI

Attributes for Explicit Binding

- [FromQuery] → Force query string
- [FromRoute] → Force route value
- [FromForm] → Force form data
- [FromBody] → Force JSON or raw body (for APIs)
- [FromHeader] → Force header

Example:

```
public IActionResult Filter([FromQuery] int page, [FromQuery] string sort)
```

4.4 Introduction to Validation

Validation ensures that incoming data is **correct and safe** before processing.
ASP.NET Core MVC provides a **server-side validation system** using **Data Annotations**.

4.4.1 Common Data Annotations

Attribute	Purpose
[Required]	Field cannot be empty
[StringLength(max, MinimumLength = min)]	Restrict length
[Range(min, max)]	Ensure numeric range
[EmailAddress]	Validate email format
[Phone]	Validate phone number
[RegularExpression]	Custom pattern matching
[Compare("OtherField")]	Compare two fields (e.g., password confirmation)

4.4.2 Example: Validating a Model

```
public class Student
{
    [Required(ErrorMessage = "Name is required")]
    [StringLength(50, MinimumLength = 3)]
    public string Name { get; set; }

    [Range(10, 100, ErrorMessage = "Age must be between 10 and 100")]
    public int Age { get; set; }

    [EmailAddress(ErrorMessage = "Invalid email format")]
    public string Email { get; set; }
}
```

Controller:


```
[HttpPost]
public IActionResult Create(Student student)
{
    if (!ModelState.IsValid)
    {
        return View(student); // Return with validation errors
    }

    // Save to database or process
    return RedirectToAction("Success");
}
```

4.5 Displaying Validation Errors in Views

Razor provides helpers to display errors:

```
<form asp-action="Create" method="post">
    <input asp-for="Name" /> //you can write regular HTML also
    <span asp-validation-for="Name" class="text-danger"></span>

    <input asp-for="Age" />
    <span asp-validation-for="Age" class="text-danger"></span>

    <input asp-for="Email" />
    <span asp-validation-for="Email" class="text-danger"></span>

    <button type="submit">Submit</button>
</form>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

- `asp-validation-for` → Displays error for the specific field
 - `_ValidationScriptsPartial` → Includes jQuery Validation for client-side validation
-

4.6 Client-Side Validation

ASP.NET Core MVC supports **unobtrusive client-side validation**:

- Automatically uses **jQuery Validation**
- Shows errors instantly without hitting the server
- Works in combination with server-side validation

Client-side validation enhances **UX**, but **server-side validation is mandatory** for security.

4.7 ModelState and Validation Workflow

1. Form or request data is posted
2. Model binding populates the model object
3. Validation attributes are checked automatically
4. `ModelState.IsValid` indicates whether the data passed validation
5. If invalid → return view with errors
6. If valid → process the data

```
if (!ModelState.IsValid)
{
    return View(student); // Show errors
}
```

4.8 Custom Validation

Sometimes, built-in attributes are not enough. You can create **custom validation attributes**.

Example: Ensuring a date is in the future

```
public class FutureDateAttribute : ValidationAttribute
{
    public override bool IsValid(object value)
    {
        if (value is DateTime date)
        {
            return date > DateTime.Now;
        }
        return false;
    }
}
```

Usage:

```
[FutureDate(ErrorMessage = "The date must be in the future")]
public DateTime StartDate { get; set; }
```

4.9 Summary of Model Binding & Validation

In this module, you learned:

- How **model binding** maps form, query, route, or JSON data to parameters
- How **validation** ensures data integrity and security
- Built-in validation attributes like [Required], [Range], [EmailAddress]
- How to use `ModelState.IsValid` in controllers
- Displaying validation errors in Razor Views
- Client-side vs server-side validation
- Creating custom validation attributes

By mastering model binding and validation, you can now safely process forms, user input, and API requests in your MVC applications.

Module 5: Custom Validation in ASP.NET Core MVC

While ASP.NET Core MVC provides **built-in validation attributes** like [Required], [Range], and [EmailAddress], real-world applications often require **custom business rules** that cannot be expressed using these standard attributes. This is where **custom validation** comes in.

Custom validation allows you to enforce **complex rules**, keep your models clean, and integrate seamlessly with the MVC validation system.

5.1 Why Use Custom Validation?

Some examples of scenarios where built-in validation is insufficient:

- Dates that must be in the future or within a specific range
- Passwords that must meet complex policies
- Unique fields that must be checked against the database
- Conditional validation (e.g., only validate if another field has a certain value)

By creating custom validation attributes, you can:

- Reuse validation logic across multiple models
 - Keep controllers and views clean
 - Ensure consistent validation rules throughout the application
-

5.2 Creating a Custom Validation Attribute

Custom validation attributes are created by inheriting from the `ValidationAttribute` class and overriding the `IsValid` method.

Example 1: Future Date Validation

```
using System;
using System.ComponentModel.DataAnnotations;

public class FutureDateAttribute : ValidationAttribute
{
    public override bool IsValid(object value)
```

```

    {
        if (value is DateTime date)
        {
            return date > DateTime.Now;
        }
        return false;
    }
}

```

Usage in a model:

```

public class Event
{
    [Required]
    public string Title { get; set; }

    [FutureDate(ErrorMessage = "Event date must be in the future")]
    public DateTime EventDate { get; set; }
}

```

Now, the EventDate property will only accept future dates.

5.3 Conditional Validation

Sometimes validation depends on another property.

Example 2: Conditional Required

```

public class ConditionalRequiredAttribute : ValidationAttribute
{
    private readonly string _dependentProperty;
    private readonly object _targetValue;

    public ConditionalRequiredAttribute(string dependentProperty, object targetValue)
    {
        _dependentProperty = dependentProperty;
        _targetValue = targetValue;
    }

    protected override ValidationResult IsValid(object value, ValidationContext
validationContext)
    {
        var property = validationContext.ObjectType.GetProperty(_dependentProperty);
        if (property == null)
            return new ValidationResult($"Unknown property: {_dependentProperty}");

        var dependentValue = property.GetValue(validationContext.ObjectInstance);

        if (dependentValue != null && dependentValue.Equals(_targetValue) && value == null)

```

```

        {
            return new ValidationResult(ErrorMessage ?? $"{validationContext.DisplayName}
is required.");
        }

        return ValidationResult.Success;
    }
}

```

Usage in a model:

```

public class Booking
{
    public bool IsInternational { get; set; }

    [ConditionalRequired("IsInternational", true, ErrorMessage = "Passport number is
required for international travel")]
    public string PassportNumber { get; set; }
}

```

Now, PassportNumber is required only if IsInternational is true.

5.4 Validation Against Database or External Sources

Sometimes, you need to validate against **existing data**, e.g., checking for duplicate email addresses.

Example 3: Unique Email Validation

```

public class UniqueEmailAttribute : ValidationAttribute
{
    protected override ValidationResult IsValid(object value, ValidationContext
validationContext)
    {
        var dbContext = (MyDbContext)validationContext.GetService(typeof(MyDbContext));
        string email = value?.ToString();

        bool exists = dbContext.Users.Any(u => u.Email == email);

        if (exists)
            return new ValidationResult(ErrorMessage ?? "Email already exists.");

        return ValidationResult.Success;
    }
}

```

Usage:

```

public class RegisterViewModel

```

```

{
    [Required]
    [EmailAddress]
    [UniqueEmail(ErrorMessage = "This email is already registered.")]
    public string Email { get; set; }

    [Required]
    public string Password { get; set; }
}

```

Note: For database-dependent validation, the attribute can receive services via **Dependency Injection** using `ValidationContext.GetService`.

5.5 Integrating Custom Validation in MVC

Custom validation attributes work seamlessly with **model binding** and **ModelState validation**.

Controller example:

```

[HttpPost]
public IActionResult Create(Event evt)
{
    if (!ModelState.IsValid)
    {
        return View(evt); // Errors displayed in Razor
    }

    _dbContext.Events.Add(evt);
    _dbContext.SaveChanges();

    return RedirectToAction("Index");
}

```

View example:

```

<form asp-action="Create" method="post">
    <input asp-for="Title" class="form-control" />
    <span asp-validation-for="Title" class="text-danger"></span>

    <input asp-for="EventDate" type="date" class="form-control" />
    <span asp-validation-for="EventDate" class="text-danger"></span>

    <button type="submit" class="btn btn-primary">Create Event</button>
</form>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}

```

5.6 Client-Side Support for Custom Validation

To enable **client-side validation** for custom attributes:

1. Implement `IClientModelValidator`
2. Register adapter in JavaScript
3. This allows **instant feedback** in forms before hitting the server

Example (advanced topic) is optional for beginners; server-side validation is always required.

5.7 Best Practices for Custom Validation

- Use attributes for reusable rules (e.g., `FutureDate`, `UniqueEmail`)
 - Keep validation logic in attributes or service classes
 - Avoid putting business logic in controllers or views
 - Use `ModelState` consistently to return validation errors
 - Combine with built-in attributes for standard checks (`Required`, `Range`, `EmailAddress`)
 - Write unit tests for custom validation attributes
-

5.8 Summary of Custom Validation

In this module, you learned:

- Why built-in validation attributes may not be enough
- How to create custom validation attributes using `ValidationAttribute`
- Conditional validation based on other property values
- Validation against external sources like databases
- How to integrate custom validation into controllers and views
- Best practices for reusable, maintainable validation

Custom validation is essential for enforcing **business rules**, maintaining **data integrity**, and providing a **better user experience**.